# ECP5 Stream Filter

***Release 1.0.0***

**HDL Team**

**Nov 01, 2026**

# Design


**1 ECP5 Stream Filter — Overview** **1**
- 1.1 Instantiation Tree . . . 1
- 1.2 Modules . . . 2

**2 top** **3**
- 2.1 top — Entity . . . 3
  - 2.1.1 Annotated Source . . . 3
- 2.2 top — Block Diagram . . . 4
  - 2.2.1 Ports . . . 4
  - 2.2.2 Interfaces . . . 4
  - 2.2.3 Signals . . . 5
  - 2.2.4 RTL Detail . . . 5
- 2.3 top — State Machine . . . 7
- 2.4 top — Timing Diagrams . . . 7
- 2.5 top — Clock Domain Analysis . . . 7
- 2.6 top — Reset Domain Analysis . . . 7
- 2.7 Synthesis Results . . . 7
- 2.8 Submodules . . . 7
  - 2.8.1 axil_regs . . . 7
    - 2.8.1.1 axil_regs — Entity . . . 7
    - 2.8.1.2 axil_regs — Block Diagram . . . 9
    - 2.8.1.3 axil_regs — State Machine . . . 13
    - 2.8.1.4 axil_regs — Timing Diagrams . . . 13
    - 2.8.1.5 axil_regs — Clock Domain Analysis . . . 13
    - 2.8.1.6 axil_regs — Reset Domain Analysis . . . 13
    - 2.8.1.7 axil_regs — Register Map . . . 13
    - 2.8.1.8 Synthesis Results . . . 14
  - 2.8.2 stream_filter . . . 14
    - 2.8.2.1 stream_filter — Entity . . . 14
    - 2.8.2.2 stream_filter — Block Diagram . . . 15
    - 2.8.2.3 stream_filter — State Machine . . . 19
    - 2.8.2.4 stream_filter — Timing Diagrams . . . 19
    - 2.8.2.5 stream_filter — Clock Domain Analysis . . . 19
    - 2.8.2.6 stream_filter — Reset Domain Analysis . . . 19
    - 2.8.2.7 Synthesis Results . . . 19

**3 Register Maps** **21**

**4 Testbenches** **23**
- 4.1 VUnit . . . 23
  - 4.1.1 tb_stream_filter . . . 23
- 4.2 cocotb . . . 23
  - 4.2.1 test_axil_regs . . . 23

4.3 Directed . . . 24
4.3.1 tb_top . . . 24

**5 Build** **25**
5.1 Summary . . . 25
5.2 Utilization . . . 25
5.2.1 Device utilisation (nextpnr) . . . 25
5.2.2 By module (Yosys) . . . 25
5.3 Timing — nextpnr . . . 25
5.4 Warnings . . . 26

**6 Indices** **27**

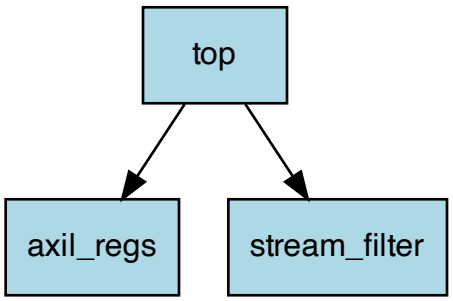
# Chapter 1

# ECP5 Stream Filter — Overview

This project contains 3 VHDL/SV modules.

Top-level module: *top*

## 1.1 Instantiation Tree



## 1.2 Modules

| Module | Source File | Description | Instantiated by | Instantiates |
|---|---|---|---|---|
| *axil_re* | `axil_reg sv` | AXI4-Lite register file for the stream filter. Four 32-bit registers: control (enable, clear), a read-only forwarded-byte | `top` | — |
| *stream_* | `stream_f sv` | Threshold filter for a byte stream. Forwards each byte that is at or above the threshold and drops the rest. | `top` | — |
| *top* | `top.sv` | ECP5 design: an AXI4-Stream threshold filter with an AXI4-Lite register file. Bytes arriving on s_axis are forwarded to m_axis if they are at or above a | *(top)* | `axil_regs`, `stream_filter` |

# Chapter 2

# top

## 2.1 top — Entity

Source file: `src/top.sv`.

ECP5 design: an AXI4-Stream threshold filter with an AXI4-Lite register file.

Bytes arriving on s_axis are forwarded to m_axis if they are at or above a threshold the bus master sets through the registers.

### 2.1.1 Annotated Source

Listing 1: src/top.sv

```
// ECP5 design: an AXI4-Stream threshold filter with an AXI4-Lite register file.
//
// Bytes arriving on s_axis are forwarded to m_axis if they are at or above a
// threshold the bus master sets through the registers.
module top (
    // System clock.
    input  logic   clk,
    // Synchronous active-high reset.
    input  logic   rst,
    // Register bus: control, status, threshold and scratch registers.
    axil_if.slave  s_axil,
    // Incoming byte stream.
    axis_if.slave  s_axis,
    // Filtered byte stream.
    axis_if.master m_axis
);
    logic        enable, clear;
    logic [7:0]  threshold;
    logic [15:0] count;

    axil_regs u_regs (
        .clk(clk), .rst(rst), .s_axil(s_axil),
        .enable(enable), .clear(clear), .threshold(threshold), .count(count)
    );

```

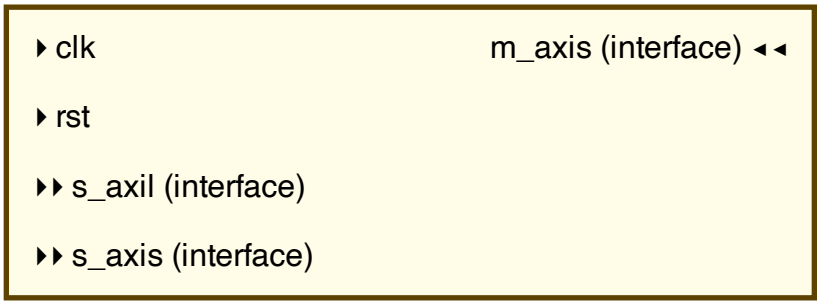
(continues on next page)

(continued from previous page)

```
    stream_filter u_filter (
        .clk(clk), .rst(rst), .enable(enable), .clear(clear), .threshold(threshold),
        .s_axis(s_axis), .m_axis(m_axis), .count(count)
    );
endmodule
```

## 2.2 top — Block Diagram

Auto-extracted from `top.sv`.




### 2.2.1 Ports

| Port | Direction | Type | Description |
|---|---|---|---|
| `clk` | `in` | `logic` | System clock. |
| `rst` | `in` | `logic` | Synchronous active-high reset. |
| `s_axil` | `slave` | `axil_if` | Register bus: control, status, threshold and scratch registers. |
| `s_axis` | `slave` | `axis_if` | Incoming byte stream. |
| `m_axis` | `master` | `axis_if` | Filtered byte stream. |

### 2.2.2 Interfaces

`axil_if` — used by `s_axil`.

| Signal | Type | slave | Description |
|---|---|---|---|
| awaddr | logic[7:0] | in | Write address. |
| awvalid | logic | in | Write address valid. |
| awready | logic | out | Write address accepted. |
| wdata | logic[31:0] | in | Write data. |
| wstrb | logic[3:0] | in | Write byte strobes. |
| wvalid | logic | in | Write data valid. |
| wready | logic | out | Write data accepted. |
| bresp | logic[1:0] | out | Write response (OKAY = 0). |
| bvalid | logic | out | Write response valid. |
| bready | logic | in | Write response accepted. |
| araddr | logic[7:0] | in | Read address. |
| arvalid | logic | in | Read address valid. |
| arready | logic | out | Read address accepted. |
| rdata | logic[31:0] | out | Read data. |
| rresp | logic[1:0] | out | Read response (OKAY = 0). |
| rvalid | logic | out | Read data valid. |
| rready | logic | in | Read data accepted. |

axis_if — used by s_axis, m_axis.

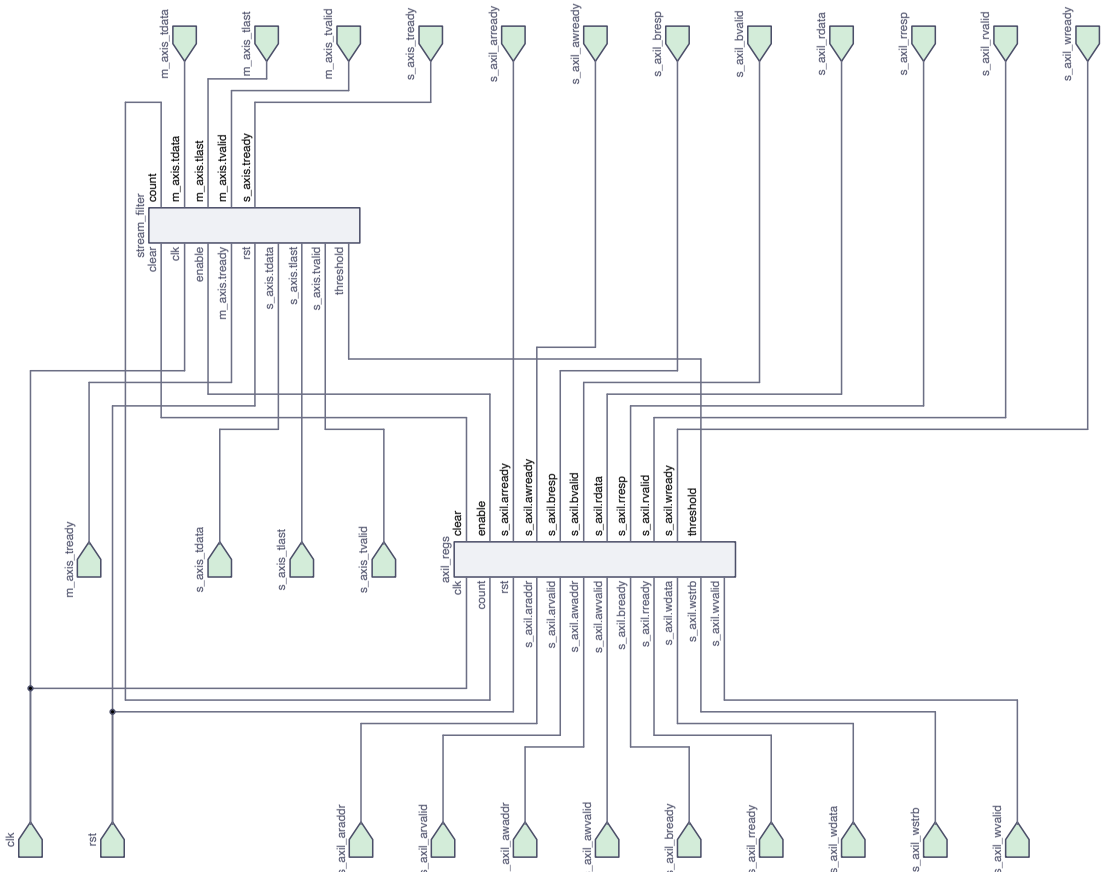
| Signal | Type | slave | master | Description |
|---|---|---|---|---|
| tdata | logic[7:0] | in | out | Payload byte. |
| tvalid | logic | in | out | The source has a byte this cycle. |
| tready | logic | out | in | The sink accepts a byte this cycle. |
| tlast | logic | in | out | Last byte of a packet. |

### 2.2.3 Signals

| Signal | Type | Description |
|---|---|---|
| enable | logic | — |
| clear | logic | — |
| threshold | logic[7:0] | — |
| count | logic[15:0] | — |

### 2.2.4 RTL Detail

Gate-level netlist generated by yosys and rendered with netlistsvg.


m_axis_tdata
m_axis_tlast
m_axis_tvalid
s_axis_tready
s_axil_arready
s_axil_awready
s_axil_bresp
s_axil_bvalid
s_axil_rdata
s_axil_rresp
s_axil_rvalid
s_axil_wready
stream_filter
count
m_axis.tdata
m_axis.tlast
m_axis.tvalid
s_axis.tready
clear
clk
enable
m_axis.tready
rst
s_axis.tdata
s_axis.tlast
s_axis.tvalid
threshold
m_axis_tready
s_axis_tdata
s_axis_tlast
s_axis_tvalid
axil_regs
clear
enable
s_axil.arready
s_axil.awready
s_axil.bresp
s_axil.bvalid
s_axil.rdata
s_axil.rresp
s_axil.rvalid
s_axil.wready
threshold
clk
count
rst
s_axil.araddr
s_axil.arvalid
s_axil.awaddr
s_axil.awvalid
s_axil.bready
s_axil.rready
s_axil.wdata
s_axil.wstrb
s_axil.wvalid
clk
rst
s_axil_araddr
s_axil_arvalid
s_axil_awaddr
s_axil_awvalid
s_axil_bready
s_axil_rready
s_axil_wdata
s_axil_wstrb
s_axil_wvalid

## 2.3 top — State Machine

No FSM detected in `top.sv`.

This module is either a structural wrapper (no combinational state logic) or uses a coding style not yet supported by the FSM extractor.

## 2.4 top — Timing Diagrams

All timing diagrams extracted from `top.sv`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

No wavedrom diagrams found in process comments.

Add `.. wavedrom::` blocks in the VHDL source above each process to have them appear here automatically.

## 2.5 top — Clock Domain Analysis

Auto-extracted from `top.sv`.

**Note**

No clocked processes detected. This module appears to be purely combinational.

## 2.6 top — Reset Domain Analysis

Auto-extracted from `top.sv`.

**Note**

No clocked processes detected. This module appears to be purely combinational.

## 2.7 Synthesis Results

| Resource | Used |
|---|---|
| LUTs | 113 |
| FFs | 156 |
| BRAMs | 0 |
| DSPs | 0 |

## 2.8 Submodules

### 2.8.1 axil_regs

#### 2.8.1.1 axil_regs — Entity

Source file: `src/axil_regs.sv`.

AXI4-Lite register file for the stream filter.

Four 32-bit registers: control (enable, clear), a read-only forwarded-byte count, the filter threshold, and a scratch register. Write strobes are ignored: every write updates the whole register. See registers/regs_axil_regs.toml.

### Annotated Source

Listing 2: src/axil_regs.sv

```
// AXI4-Lite register file for the stream filter.
//
// Four 32-bit registers: control (enable, clear), a read-only forwarded-byte
// count, the filter threshold, and a scratch register. Write strobes are
// ignored: every write updates the whole register. See registers/regs_axil_regs.toml.
module axil_regs (
    // System clock.
    input  logic        clk,
    // Synchronous active-high reset.
    input  logic        rst,
    // Register access from the bus master.
    axil_if.slave       s_axil,
    // Enables the stream filter.
    output logic        enable,
    // Holds the forwarded-byte counter at zero while high.
    output logic        clear,
    // Smallest byte value the filter forwards.
    output logic [7:0]  threshold,
    // Bytes forwarded since the counter was last cleared.
    input  logic [15:0] count
);
    logic [31:0] ctrl_r, threshold_r, scratch_r, rdata_r;
    logic        bvalid_r, rvalid_r;

    // Interface members are copied to plain signals first: Yosys cannot take a bit-select
    // of an interface member inside a submodule.
    wire [7:0]  awaddr = s_axil.awaddr;
    wire [7:0]  araddr = s_axil.araddr;
    wire [31:0] wdata  = s_axil.wdata;

    // A write is taken when both address and data are present and no response is pending.
    wire wr_fire = s_axil.awvalid && s_axil.wvalid && !bvalid_r;
    wire rd_fire = s_axil.arvalid && !rvalid_r;

    always_ff @(posedge clk) begin
        if (rst) begin
            ctrl_r      <= 32'd0;
            threshold_r <= 32'd0;
            scratch_r   <= 32'd0;
            rdata_r     <= 32'd0;
            bvalid_r    <= 1'b0;
            rvalid_r    <= 1'b0;
        end else begin
            if (wr_fire) begin
```

(continues on next page)

(continued from previous page)

```
                bvalid_r <= 1'b1;
                case (awaddr[3:2])
                    2'd0:    ctrl_r      <= wdata;
                    2'd2:    threshold_r <= wdata;
                    2'd3:    scratch_r   <= wdata;
                    default: ;
                endcase
            end else if (s_axil.bready) begin
                bvalid_r <= 1'b0;
            end

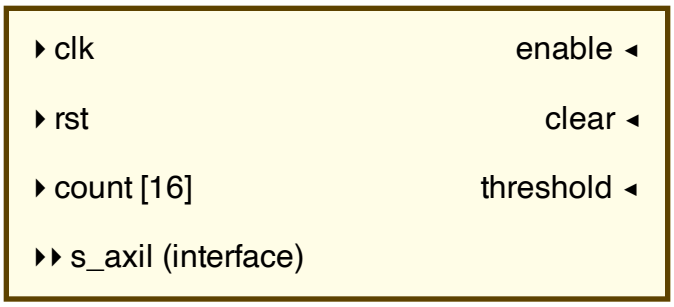
            if (rd_fire) begin
                rvalid_r <= 1'b1;
                case (araddr[3:2])
                    2'd0:    rdata_r <= ctrl_r;
                    2'd1:    rdata_r <= {16'd0, count};
                    2'd2:    rdata_r <= threshold_r;
                    default: rdata_r <= scratch_r;
                endcase
            end else if (s_axil.rready) begin
                rvalid_r <= 1'b0;
            end
        end
    end

    assign s_axil.awready = wr_fire;
    assign s_axil.wready  = wr_fire;
    assign s_axil.bresp   = 2'b00;
    assign s_axil.bvalid  = bvalid_r;
    assign s_axil.arready = rd_fire;
    assign s_axil.rdata   = rdata_r;
    assign s_axil.rresp   = 2'b00;
    assign s_axil.rvalid  = rvalid_r;

    assign enable    = ctrl_r[0];
    assign clear     = ctrl_r[1];
    assign threshold = threshold_r[7:0];
endmodule
```

#### 2.8.1.2 axil_regs — Block Diagram

Auto-extracted from `axil_regs.sv`.




### Ports

| Port | Direction | Type | Description |
|---|---|---|---|
| clk | in | logic | System clock. |
| rst | in | logic | Synchronous active-high reset. |
| s_axil | slave | axil_if | Register access from the bus master. |
| enable | out | logic | Enables the stream filter. |
| clear | out | logic | Holds the forwarded-byte counter at zero while high. |
| threshold | out | logic[7:0] | Smallest byte value the filter forwards. |
| count | in | logic[15:0] | Bytes forwarded since the counter was last cleared. |

### Interfaces

axil_if — used by s_axil.

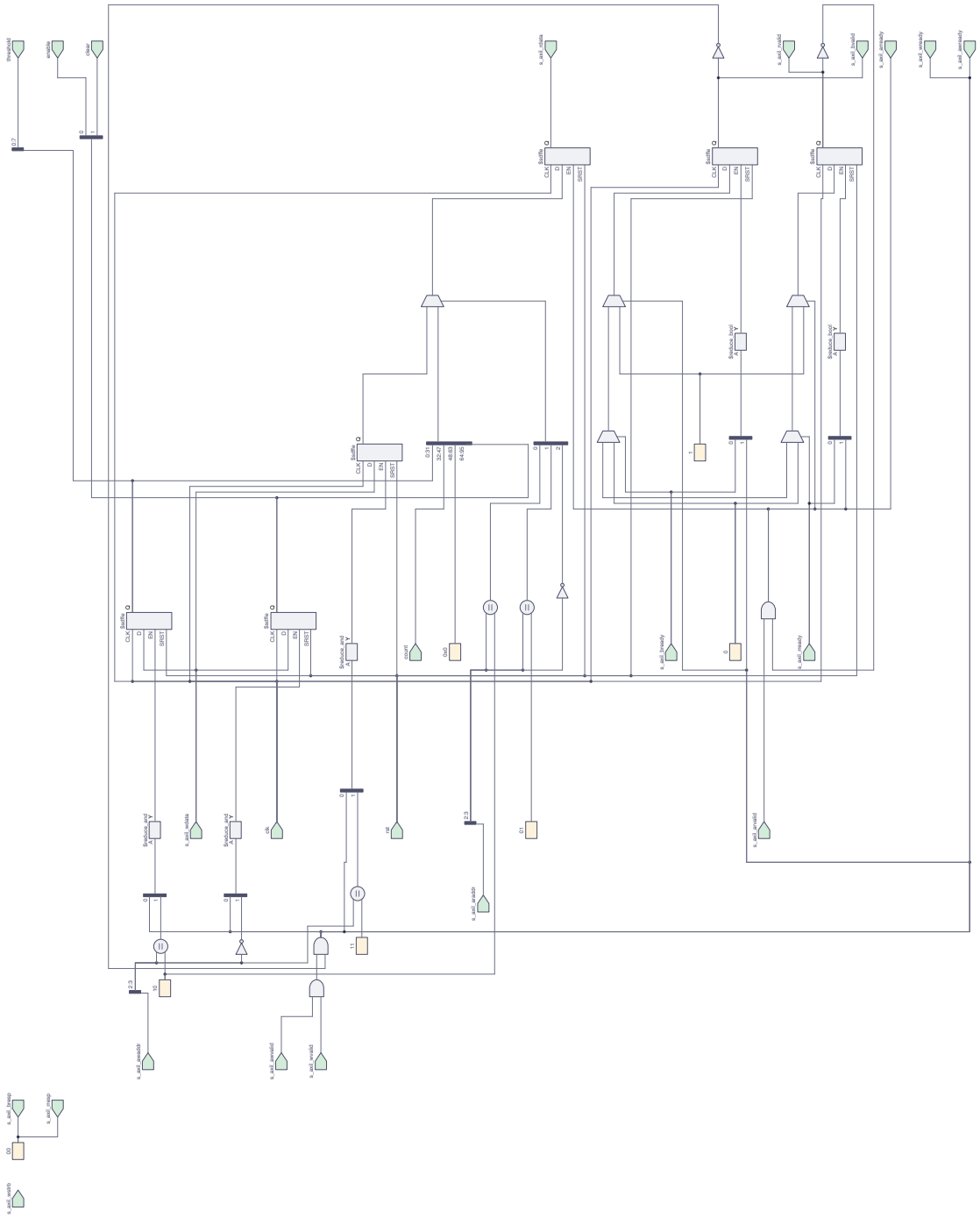
| Signal | Type | slave | Description |
|---|---|---|---|
| awaddr | logic[7:0] | in | Write address. |
| awvalid | logic | in | Write address valid. |
| awready | logic | out | Write address accepted. |
| wdata | logic[31:0] | in | Write data. |
| wstrb | logic[3:0] | in | Write byte strobes. |
| wvalid | logic | in | Write data valid. |
| wready | logic | out | Write data accepted. |
| bresp | logic[1:0] | out | Write response (OKAY = 0). |
| bvalid | logic | out | Write response valid. |
| bready | logic | in | Write response accepted. |
| araddr | logic[7:0] | in | Read address. |
| arvalid | logic | in | Read address valid. |
| arready | logic | out | Read address accepted. |
| rdata | logic[31:0] | out | Read data. |
| rresp | logic[1:0] | out | Read response (OKAY = 0). |
| rvalid | logic | out | Read data valid. |
| rready | logic | in | Read data accepted. |

### Signals

| Signal | Type | Description |
|---|---|---|
| ctrl_r | logic[31:0] | — |
| threshold_r | logic[31:0] | — |
| scratch_r | logic[31:0] | — |
| rdata_r | logic[31:0] | — |
| bvalid_r | logic | — |
| rvalid_r | logic | — |

### RTL Detail

Gate-level netlist generated by yosys and rendered with netlistsvg.



#### 2.8.1.3 axil_regs — State Machine

No FSM detected in `axil_regs.sv`.

This module is either a structural wrapper (no combinational state logic) or uses a coding style not yet supported by the FSM extractor.

#### 2.8.1.4 axil_regs — Timing Diagrams

All timing diagrams extracted from `axil_regs.sv`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

No wavedrom diagrams found in process comments.

Add `.. wavedrom::` blocks in the VHDL source above each process to have them appear here automatically.

#### 2.8.1.5 axil_regs — Clock Domain Analysis

Auto-extracted from `axil_regs.sv`.

**Note**

No clocked processes detected. This module appears to be purely combinational.

#### 2.8.1.6 axil_regs — Reset Domain Analysis

Auto-extracted from `axil_regs.sv`.

**Note**

All clocked processes use a single reset domain (`rst`, sync). No reset domain crossings detected.

##### Reset Domain

| Process | Clock | Reset | Style |
|---|---|---|---|
| 2 | `clk` | `rst` | Sync |

#### 2.8.1.7 axil_regs — Register Map

| Register | Offset | Access | Description |
|---|---|---|---|
| ctrl | 0x00 | r/w | Control register for the stream filter. |
| status | 0x04 | r | Filter status. |
| threshold | 0x08 | r/w | Filter threshold. |
| scratch | 0x0C | r/w | General-purpose scratch register. Reads back what was written; no effect on the design. |

#### ctrl (0x00)

| Field | Type | Default | Description |
|---|---|---|---|
| `enable` | bit | 0 | Assert to let the filter forward bytes. While low every byte is dropped. |
| `clear` | bit | 0 | Hold high to hold the forwarded-byte counter at zero. |

#### status (0x04)

| Field | Type | Default | Description |
|---|---|---|---|
| `forwarded` | bit_vector(16) | | Bytes forwarded since the counter was last cleared. |

#### threshold (0x08)

| Field | Type | Default | Description |
|---|---|---|---|
| `value` | bit_vector(8) | 00000000 | Smallest byte value that is forwarded. Bytes below it are dropped. |

#### scratch (0x0C)

| Field | Type | Default | Description |
|---|---|---|---|
| `value` | bit_vector(32) | 00000000000000000( | Software scratch value. |

#### 2.8.1.8 Synthesis Results

*No synthesis data available for this module.*

### 2.8.2 stream_filter

#### 2.8.2.1 stream_filter — Entity

Source file: `src/stream_filter.sv`.

Threshold filter for a byte stream.

Forwards each byte that is at or above the threshold and drops the rest. One register stage; counts the bytes it forwards.

#### Annotated Source

Listing 3: src/stream_filter.sv

```
1  // Threshold filter for a byte stream.
2  //
3  // Forwards each byte that is at or above the threshold and drops the rest.
4  // One register stage; counts the bytes it forwards.
5  module stream_filter (
```

(continues on next page)

(continued from previous page)

```
    // System clock.
    input  logic        clk,
    // Synchronous active-high reset.
    input  logic        rst,
    // Filtering is on while high; while low every byte is dropped.
    input  logic        enable,
    // Holds the counter at zero while high.
    input  logic        clear,
    // Smallest byte value that is forwarded.
    input  logic [7:0]  threshold,
    // Incoming bytes.
    axis_if.slave       s_axis,
    // Forwarded bytes.
    axis_if.master      m_axis,
    // Bytes forwarded since the counter was last cleared.
    output logic [15:0] count
);
    logic [7:0] tdata_r;
    logic       tvalid_r, tlast_r;

    // The output register is free when empty or being read this cycle.
    wire  can_load = !tvalid_r || m_axis.tready;
    wire  pass     = s_axis.tvalid && enable && (s_axis.tdata >= threshold);

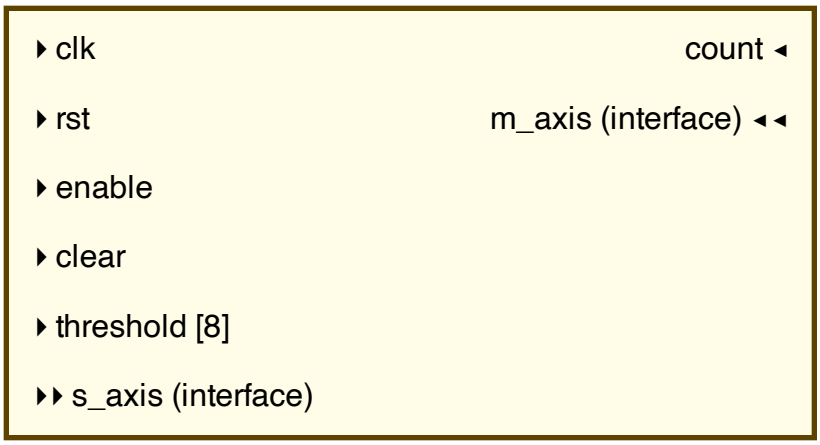
    always_ff @(posedge clk) begin
        if (rst || clear) begin
            count <= 16'd0;
        end else if (can_load && pass) begin
            count <= count + 16'd1;
        end

        if (rst) begin
            tvalid_r <= 1'b0;
        end else if (can_load) begin
            tvalid_r <= pass;
            tdata_r  <= s_axis.tdata;
            tlast_r  <= s_axis.tlast;
        end
    end

    assign s_axis.tready = can_load;
    assign m_axis.tdata  = tdata_r;
    assign m_axis.tvalid = tvalid_r;
    assign m_axis.tlast  = tlast_r;
endmodule
```

#### 2.8.2.2 stream_filter — Block Diagram

Auto-extracted from `stream_filter.sv`.




### Ports

| Port | Direction | Type | Description |
|---|---|---|---|
| `clk` | `in` | `logic` | System clock. |
| `rst` | `in` | `logic` | Synchronous active-high reset. |
| `enable` | `in` | `logic` | Filtering is on while high; while low every byte is dropped. |
| `clear` | `in` | `logic` | Holds the counter at zero while high. |
| `threshold` | `in` | `logic[7:0]` | Smallest byte value that is forwarded. |
| `s_axis` | `slave` | `axis_if` | Incoming bytes. |
| `m_axis` | `master` | `axis_if` | Forwarded bytes. |
| `count` | `out` | `logic[15:0]` | Bytes forwarded since the counter was last cleared. |

### Interfaces

`axis_if` — used by `s_axis`, `m_axis`.

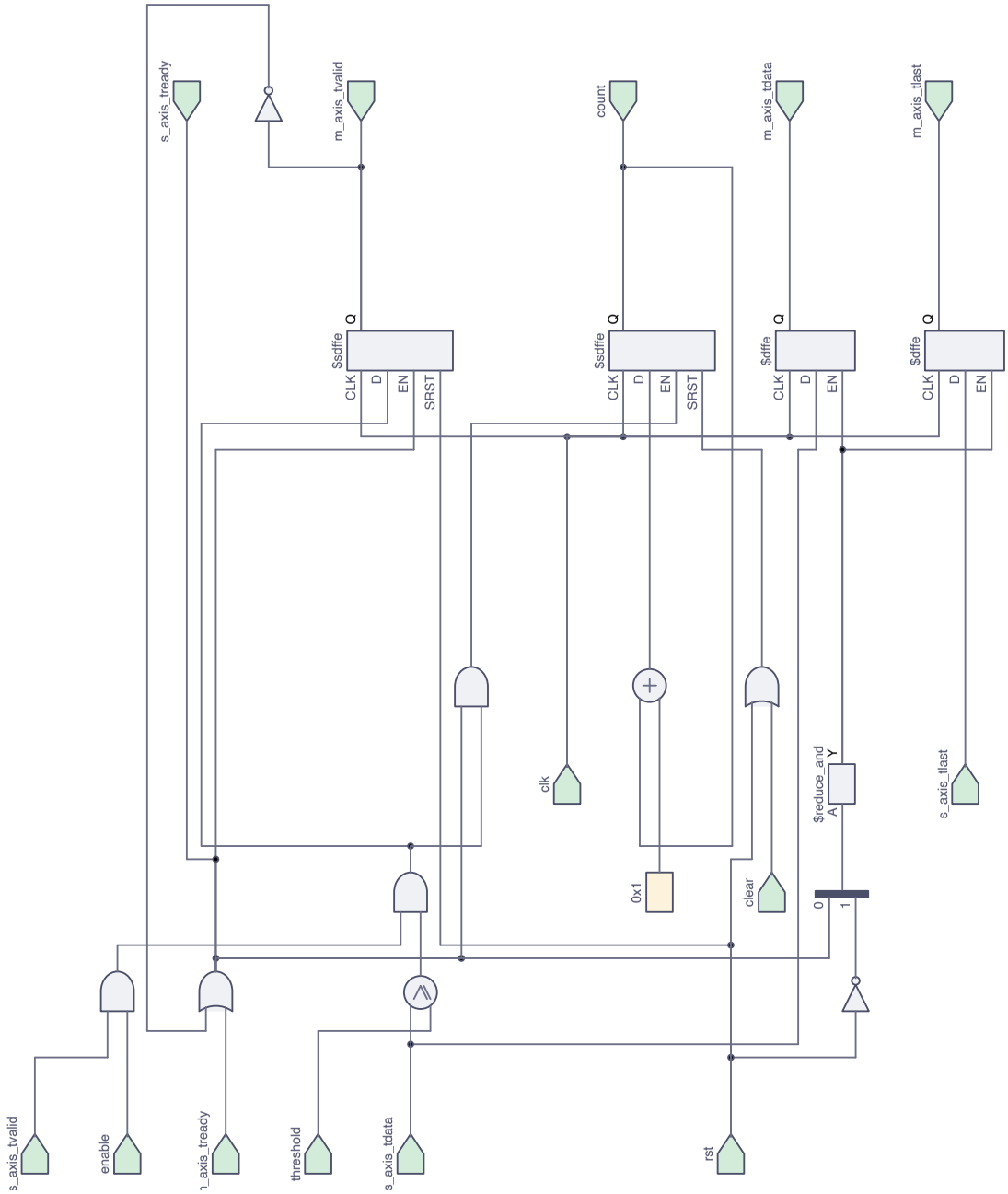
| Signal | Type | slave | master | Description |
|---|---|---|---|---|
| `tdata` | `logic[7:0]` | in | out | Payload byte. |
| `tvalid` | `logic` | in | out | The source has a byte this cycle. |
| `tready` | `logic` | out | in | The sink accepts a byte this cycle. |
| `tlast` | `logic` | in | out | Last byte of a packet. |

### Signals

| Signal | Type | Description |
|---|---|---|
| `tdata_r` | `logic[7:0]` | — |
| `tvalid_r` | `logic` | — |
| `tlast_r` | `logic` | — |

### RTL Detail

Gate-level netlist generated by `yosys` and rendered with `netlistsvg`.


s_axis_tready
m_axis_tvalid
count
m_axis_tdata
m_axis_tlast
$sdffe
$sdffe
$dffe
$dffe
Q
CLK
D
EN
SRST
clk
0x1
clear
$reduce_and
A
Y
0
1
s_axis_tlast
s_axis_tvalid
enable
m_axis_tready
threshold
s_axis_tdata
rst

#### 2.8.2.3 stream_filter — State Machine

No FSM detected in `stream_filter.sv`.

This module is either a structural wrapper (no combinational state logic) or uses a coding style not yet supported by the FSM extractor.

#### 2.8.2.4 stream_filter — Timing Diagrams

All timing diagrams extracted from `stream_filter.sv`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

No wavedrom diagrams found in process comments.

Add `.. wavedrom::` blocks in the VHDL source above each process to have them appear here automatically.

#### 2.8.2.5 stream_filter — Clock Domain Analysis

Auto-extracted from `stream_filter.sv`.

**Note**

No clocked processes detected. This module appears to be purely combinational.

#### 2.8.2.6 stream_filter — Reset Domain Analysis

Auto-extracted from `stream_filter.sv`.

**Note**

No clocked processes detected. This module appears to be purely combinational.

#### 2.8.2.7 Synthesis Results

*No synthesis data available for this module.*

# Chapter 3

# Register Maps

# Chapter 4

# Testbenches

| Framework | Testbenches | Tests |
|---|---|---|
| VUnit | 1 | 3 |
| cocotb | 1 | 3 (1 skipped) |
| Directed | 1 | 2 |
| **Total** | 3 | 8 (1 skipped) |

## 4.1 VUnit

### 4.1.1 tb_stream_filter

**File**
`tb/vunit/tb_stream_filter.sv`

**Tests module**
*stream_filter*

**Purpose**
Testbench for stream_filter. Drives bytes into s_axis and checks which ones come out of m_axis: the filter forwards bytes at or above the threshold and drops the rest.

**Tests**
3 tests

| Test | Purpose |
|---|---|
| `passes_at_or_above_threshold` | A byte equal to or above the threshold appears on m_axis. |
| `drops_below_threshold` | A byte below the threshold is dropped. |
| `counts_forwarded_bytes` | The counter counts only the bytes that were forwarded. |

## 4.2 cocotb

### 4.2.1 test_axil_regs

**File**
`tb/cocotb/test_axil_regs.py`

**Tests module**
*axil_regs*

**Purpose**
Register file tests for axil_regs.

**Tests**
3 tests, 1 skipped

| Test | Purpose |
|---|---|
| `reset_values` | Every register reads back zero after reset. |
| `scratch_write_read` | A value written to the scratch register reads back unchanged. |
| `threshold_drives_filter` *(skipped)* | The threshold register changes which bytes the filter forwards (needs the full top level). |

## 4.3 Directed

### 4.3.1 tb_top

**File**
`tb/directed/tb_top.sv`

**Tests module**
*top*

**Purpose**
Directed testbench for the top level. Programs the filter through the AXI4-Lite bus, then streams bytes through it, with no verification framework: each scenario below is marked with a TEST comment.

**Tests**
2 tests

| Test | Purpose |
|---|---|
| `register_write_read` | a value written to scratch reads back unchanged |
| `filter_pass_through` | bytes at or above the threshold reach m_axis, others do not |

# Chapter 5

# Build

## 5.1 Summary

| | |
|---|---|
| Yosys | 0.56+186 |
| Device | ECP5 25k class |
| Reports generated | 2026-10-31 22:35 |
| Documented at | v3.12.0-15-gd3eb583 |

## 5.2 Utilization

### 5.2.1 Device utilisation (nextpnr)

| Resource | Used | Available | % |
|---|---|---|---|
| TRELLIS_IO | 122 | 197 | 61 |
| DCCA | 1 | 56 | 1 |
| TRELLIS_FF | 156 | 24,288 | 0 |
| TRELLIS_COMB | 147 | 24,288 | 0 |

24 other resource types unused.

### 5.2.2 By module (Yosys)

| Module | LUTs | FFs | BRAMs | DSPs |
|---|---|---|---|---|
| top | 113 | 156 | 0 | 0 |

## 5.3 Timing — nextpnr

| Clock | Fmax (MHz) | Target (MHz) | Status |
|---|---|---|---|
| clk | 220.41 | 100.0 | ✓ PASS |

## 5.4 Warnings

146 warnings in the build logs; the most frequent:

| Message | Count |
|---|---|
| IO '...' is unconstrained in LPF and will be automatically placed | 121 |
| Identifier '...' is implicitly declared. | 24 |
| The network is combinational. | 1 |

# Chapter 6

# Indices

- genindex
- search